```
#include <stdio.h>
#include <limits.h>

#define INF INT_MAX
#define V 100 // Maximum number of vertices

// Function to find the vertex with minimum distance value
int minDistance(int dist[], int visited[], int n) {
    int min = INF, min_index;

    for (int v = 0; v < n; v++) {
        if (visited[v] == 0 && dist[v] <= min) {
            min = dist[v], min_index = v;
        }
    }

    return min_index;
}

// Dijkstra's algorithm function
void dijkstra(int graph[V][V], int n, int src) {
    int dist[V];  // Output array to hold the shortest distance from src to each vertex
    int visited[V]; // visited[i] will be 1 if vertex i is included in shortest path tree

    // Initialize distances to INF and visited array to 0
    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }

    // Distance of source vertex from itself is always 0
    dist[src] = 0;

    // Find the shortest path for all vertices
    for (int count = 0; count < n - 1; count++) {
        // Pick the minimum distance vertex from the set of vertices not yet processed
        int u = minDistance(dist, visited, n);

        // Mark the picked vertex as processed
        visited[u] = 1;

        // Update dist value of the adjacent vertices of the picked vertex
        for (int v = 0; v < n; v++) {
            // Update dist[v] only if there is an edge from u to v, and
            // the vertex v is not visited, and the total weight of the path
            // from src to v through u is smaller than the current value of dist[v]
            if (!visited[v] && graph[u][v] && dist[u] != INF && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    // Print the calculated shortest distances
    printf("Vertex\tDistance from Source\n");
    for (int i = 0; i < n; i++) {
        if (dist[i] == INF)
```

```
            printf("%d\t\tINF\n", i);
        else
            printf("%d\t\t%d\n", i, dist[i]);
    }
}

int main() {
    int n, src;
    int graph[V][V];

    // Input the number of vertices
    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    // Input the adjacency matrix
    printf("Enter the adjacency matrix (use 0 for no direct edge):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
            // Replace 0s with INF for non-diagonal elements (if there's no edge)
            if (graph[i][j] == 0 && i != j) {
                graph[i][j] = INF;
            }
        }
    }

    // Input the source vertex
    printf("Enter the source vertex: ");
    scanf("%d", &src);

    // Call Dijkstra's algorithm
    dijkstra(graph, n, src);

    return 0;
}
```